

Software Testing 2025/6 Portfolio

Arsenii Harbar — s2529401

January 19, 2026

Repository: <https://github.com/zlnare/flash-crash-sentinel> (read access granted for coursework assessment)

1 Outline of the Software Being Tested

RiskBeacon is a Python-based real-time market risk monitoring system (FastAPI backend + Streamlit dashboard) that ingests OHLCV ticks (POST `/api/tick`), computes quantitative risk metrics (VPIN, Yang–Zhang volatility, regime classification), persists results to per-ticker CSVs, and generates dashboard-ready Plotly HTML artifacts (POST `/api/plots/generate`). The system is high-risk because errors can misrepresent market stress; therefore the test process targets quantitative invariants, robustness to malformed data, external dependency failures (Interactive Brokers), and performance under load. Requirements and traceability are documented in `docs/requirements.md` and `docs/requirements_to_tests.md`.

2 Learning Outcomes

1. **Analyze requirements to determine appropriate testing strategies** [20%]
 - (a) **Range of requirements, functional requirements, measurable quality attributes, qualitative requirements**
Range: 17 testable requirements (RB-REQ-01..RB-REQ-17) spanning functional correctness (tick persistence, plot generation), quantitative invariants (VPIN in $[0, 1]$, volatility ≥ 0), robustness (bad CSV lines, missing sentiment, IB failure), performance targets (latency/time), and MLOps reproducibility (MLflow artifacts). Evidence: `docs/requirements.md`.
 - (b) **Level of requirements, system, integration, unit**
Levels: Requirements span unit (RB-REQ-06/07/09), integration (RB-REQ-02/13), and system (RB-REQ-01/03/04/05/14/15/16/17). Evidence: `docs/requirements.md` (each requirement includes a level) and traceability: `docs/requirements_to_tests.md`.
 - (c) **Identifying test approach for chosen attributes**
Testing philosophy: Verification (requirements-driven tests + automated checks: pytest, GX, MLflow, perf) vs validation (dashboard fitness-for-purpose). Optimistic vs pessimistic inaccuracy mitigated via redundancy and restriction. **A&T principles:** Sensitivity (unit tests: `test/unit_testing/service_modules/*`), Redundancy (VPIN/vol: Hypothesis invariants + GX + integration), Restriction (FinBERT disabled in CI via `RISKBEACON_DISABLE_FINBERT=true`), Partition (category-partitioning for `/api/tick` inputs), Visibility (CI artifacts, perf prints), Feedback (defect-driven evolution: commits `0d5997f`, `c1cae2d`, `ed7f133`).
 - (d) **Assess the appropriateness of your chosen testing approach**
Appropriateness: Approaches match risk profile: invariants for oracle-free quant correctness, integration for API/CSV side-effects, perf for latency targets, CI for

repeatability. Limitations (IB permissions/market hours; GX Windows toolchain) documented and mitigated (`ib_troubleshoot.py`, pinned `requirements-dev.txt`).

2. **Design and implement comprehensive test plans with instrumented code** [20%]

(a) **Construction of the test plan**

Plan: `docs/PLAN.md` defines scope, test inventory (tests → files → RB-REQs), risk register (IB, malformed CSVs, heavy deps), and monitoring metrics (coverage trend, CI pass rate, perf med/p95).

(b) **Evaluation of the quality of the test plan**

Quality: Plan prioritises critical path (ingestion → persistence → plots) early. Residual gaps explicitly stated (metamorphic ML tests), preventing over-claiming.

(c) **Instrumentation of the code**

Instrumentation: Added logging and controllable behaviour: plot loader deduplication (`services/plotService.py`), volatility cache (`services/vol_service.py`), CI-safe sentiment flag (`services/sentimentService.py`), IB status endpoints + troubleshoot tool (`ib_troubleshoot.py`).

(d) **Evaluation of the instrumentation**

Evaluation: Instrumentation improves visibility and feedback (CI artifacts, logs, deterministic harnesses). IB variability mitigated via graceful degradation and diagnostics.

3. **Apply a wide variety of testing techniques and compute test coverage and yield according to a variety of criteria** [25%]

(a) **Range of techniques**

Techniques: Property-based invariants (`test/unit_testing/Quant_services/*`), unit tests (`test/unit_testing/service_modules/*`), integration (FastAPI Test-Client: `test/integration_testing/test_api_tick_persists.csv.py`, `test_api_plot_generation.py`), performance measurement (`test/integration_testing/test_perf*`), data validation (GX: `test/data_validation/*`), MLOps (MLflow: `test/mlops/test_mlflow_tracking.py`), backtesting (`test/backtesting/research.ipynb` with train/test splits, sentiment integration, stop-loss, multi-asset support). **Category-partition for `/api/tick`:** Timestamp (ISO8601; invalid; timezone-aware/naive), Volume (< 0 , $= 0$, typical, huge), OHLC (ordered; missing; extreme returns), Ticker (normal; lowercase; empty).

(b) **Evaluation criteria for the adequacy of the testing**

Adequacy criteria: Coverage (pytest-cov) and requirement traceability (RB-REQ → tests). Mutation testing (mutmut) on `services/vpin_service.py` via `.github/workflows/mutation.yml` generates 109+ mutants; stats collection faces dependency isolation challenges in mutmut sandbox. **Results:** 200 passed, 3 skipped, 4 xfailed tests. CI uploads artifacts (coverage + GX suites). xfailed tests capture discovered imperfections (e.g., plot filename/endpoint mismatch) as executable checks. **Backtesting:** AMD/NVDA/TSLA portfolio (2023-12 to 2025-11), test period (2025-04 to 2025-11) achieved 11.15% return with Sharpe 0.82. Evidence: `test/backtesting/research.ipynb` with train/test splits, sentiment filters, risk management.

(c) **Evaluation of the results**

Evaluation: Coverage treated as adequacy criterion (not proof). Low-coverage modules justified (external artifacts, optional integrations). Performance compared to RB-REQ-16/17 targets, regressions prevented via CI. Backtesting demonstrates sound methodology (train/test split, fees, no look-ahead bias). Test period (11.15% return, Sharpe 0.82) validates out-of-sample reliability. Validates RB-REQ-11 (time-series split enforcement).

4. Evaluate the limitations of a given testing process, using statistical methods where appropriate, and summarise outcomes [15%]

(a) Identifying gaps and omissions in the testing process

Gaps: (1) Regime metamorphic tests (RB-REQ-10) planned but not implemented; (2) Backtesting (RB-REQ-11) implemented (`test/backtesting/research.ipynb`); (3) IB live-market testing limited by permissions/market hours (justified: `services/ib_client_service.py` at 46% coverage); (4) Some error paths in `services/vol_service.py` Yang-Zhang (lines 206-310) uncovered due to date mapping complexity. Documented in `docs/PLAN.md`.

(b) Identifying target coverage/performance levels for the different testing procedures

Targets: Performance targets defined in RB-REQ-16/17 (`docs/requirements.md`). Coverage monitored via CI; mutation testing planned for high-risk quant services.

(c) Discussing how the testing carried out compares with the target levels
Measured evidence (captured from local test output):

Metric	Target	Measured
/api/tick median/p95	< 50ms / < 150ms	14.16ms / 19.83ms (n=75)
Plot generation time (3000 rows)	< 300s	0.82s (3000 rows)
Coverage (<code>api/controllers/services</code>)	tracked	76% local / 75% CI (pytest-cov)

(d) Discussion of what would be necessary to achieve the target level

Achievement: Coverage 76% overall (local), `api/routes.py` 80%, `services/sentimentService.py` 86% (up from 61%), `controllers/ServiceController.py` 67%, `services/vpin_service.py` 96%. **To reach 80%:** target gaps in `services/vol_service.py` (66%), `services/hmm_regime_service.py` (65%), `controllers/ServiceController.py` (67%) with additional edge case/error path tests. Extend stress tests; add contract tests with mocks.

5. Conduct reviews, inspections, and design and implement automated testing processes [20%]

(a) Identify and apply review criteria to selected parts of the code and identify issues in the code

Review criteria: Correctness (invariants, edge cases), robustness (bad inputs/external failures), maintainability (testability, logging, dependency isolation). **Fixes:** Volatility cache initialisation (commit `ed7f133`), unit-test package shadowing (commit `876b1d0`), CI/GX suite export hardening (commits `0d5997f..0f4aad2`). **Imperfections:** Plot generation produces ticker-prefixed filenames while endpoints accept non-prefixed names; captured as `xfail` tests in `test/integration_testing/test_api_routes_more_coverage.py`.

(b) Construct an appropriate CI pipeline for the software CI pipeline:

GitHub Actions (`.github/workflows/ci.yml`) runs pytest with coverage, exports GX suites, uploads `coverage.xml`, `htmlcov/`, `docs/data_validation/gx_suites/`.

(c) Automate some aspects of the testing

Automation: Automate repetitive execution (unit/integration/perf, coverage, suite exports) while restricting heavy dependencies (FinBERT disabled in CI; IB not required) for cost-effectiveness and visibility.

(d) Demonstrate the CI pipeline functions as expected

Evidence: Green CI runs + downloadable artifacts. Local reproducibility: `docs/how_to_run.md`.